

Optimization Against AI Slop: Three Empirical Workstreams from the PromptLab Repository

PromptLab / STAT 4830 Repository Synthesis

May 14, 2026

Abstract

This report synthesizes the three main research threads implemented in the PromptLab repository: (1) a Pangram-based optimization tournament that compares multiple deslopification strategies under a shared evaluation harness, (2) a direct reinforcement-learning deslopifier trained with REINFORCE against EditLens, and (3) a two-stage “Promptlab-v2” pipeline that uses evolutionary prompt search to generate contrastive rewrite pairs and then distills them into a lightweight T5 rewriter. Taken together, these workstreams show that AI-detection evasion is indeed an optimization problem, but that the dominant challenge is not merely reducing a detector score. The harder problem is reducing that score while preserving semantics, length, fluency, and output validity. The tournament infrastructure demonstrates why a common benchmark is necessary: most entrants either worsened the detector score or appeared to improve only by violating similarity and length expectations. The RL deslopifier achieved the clearest EditLens-grounded gain, lowering mean EditLens from 0.999 to 0.670 and moving 45% of essays out of the fully-AI bucket, but it also exposed classic reward-hacking behavior on a subset of inputs. The Promptlab-v2 branch achieved the strongest detector-specific result, reducing mean slop from 0.391 to 0.0998 on held-out validation pairs relative to a v1 prompt-mode rewriting baseline, but with a hard architectural ceiling on output length imposed by the T5-base backbone. Qualitative analysis of the T5 rewriter reveals that slop reduction and length preservation are in direct tension under this architecture. When the model operates within its natural stopping range—as on the vaccination topic, where it achieves a slop score of 0.077 against a 0.916 baseline—compression is a feature: the output is coherent, structurally transformed, and genuinely less detectable. When the same model is forced to match input length via `min_length`, it fills the required tokens with repetitive looping rather than new content, as observed on the CSR topic. This failure mode is not a tunable parameter—it is a consequence of T5-base’s 512-token context ceiling and summarization pretraining bias, and points directly at decoder-only architectures with explicit length rewards as the necessary next step. The overall lesson is therefore nuanced: detector-guided optimization works, yet its validity depends on the reward definition, the judge family, and the structural constraints imposed during search or training.

1 Introduction

The PromptLab repository studies what the team informally calls *AI slop*: text that is grammatical and often superficially competent, but still carries recognizable stylistic artifacts of machine generation. Rather than treating this only as a detection problem, the repository frames it as an optimization problem. The objective is to make text less detectably machine-generated, while constraining the rewrite process so that the result remains semantically faithful, fluent, and non-degenerate.

That framing produces an immediate tension. If the optimization target is only a detector score, the easiest solutions are often not the most meaningful ones. A model can shorten text, refuse to continue, repeat odd token patterns, or otherwise exploit detector blind spots without learning a human-like editing style. Much of the scientific value of this repository comes from confronting that tension directly. Across the codebase, detector-guided rewriting is explored through three different optimization paradigms:

1. a shared Pangram tournament that standardizes datasets, judges, and qualification gates;
2. online policy optimization with REINFORCE using EditLens as a continuous reward;
3. offline search-and-distillation, where an evolutionary prompt optimizer generates contrastive rewrite pairs for supervised training of a lightweight T5-base rewriter.

These workstreams do not share one perfectly unified metric regime. Early and late branches use different detectors, and some artifacts are directly comparable while others are only conceptually comparable. This report therefore treats the repository not as a single leaderboard, but as a coherent empirical program on detector-aware text optimization.

Before these three mature workstreams, the repository also contains a preliminary verifier stage. A Kaggle essay verifier notebook reports a DistilBERT-based classifier with test accuracy 0.993 and ROC-AUC 1.000 on its local task, while later experiment specifications explicitly note that such near-perfect performance was likely inflated by easy or dataset-specific artifacts. That early result is still important: it motivated the shift from simply classifying slop toward optimizing against it, and it foreshadowed the need for harder judges such as Pangram EditLens.

2 Scope and Evaluation Regimes

Any academic reading of the repository must begin with a measurement warning: the three main workstreams are not scored on one universal axis.

The Pangram tournament and the RL deslopifier both operate in the EditLens family, but even there the tournament uses a fast Pangram RoBERTa scorer during development while reserving Pangram 3B as the official judge. The Promptlab-v2 branch uses a local *SlopDetector* score built on `editlens/roberta-large`—the same underlying model family as EditLens, though a smaller variant than the Pangram 3B judge used as the tournament’s official evaluator. This means that a number such as 0.0998 in Promptlab-v2 should not be read as directly superior to a number such as 0.670 in the RL pipeline: the former is a within-detector improvement on a small, curated validation set evaluated against RoBERTa-large, while the latter is a result on held-out Pangram data against a different score scale and model size.

The repository is most coherent when viewed through common scientific questions rather than raw metric values:

- Can detector-aware optimization reduce detectability at all?
- What constraints are required to prevent reward hacking?
- Is online RL necessary, or can offline data generation and supervised learning recover the same effect?
- How much do conclusions depend on the judge family and evaluation harness?

Workstream	Core method	Best result	Key caveat
Pangram tournament	Shared benchmark over LoRA, SFT, REINFORCE, and DPO variants under common similarity and length gates.	No robust winner; M4 achieves only +0.00035 mean delta in phase 2, with 16% pass rate and 0.598 median length ratio.	Once fidelity constraints are enforced, most apparent gains disappear.
RL deslopifier	Llama 3.2 3B Instruct with LoRA, optimized by REINFORCE against EditLens plus fluency and length regularization.	Mean EditLens falls 0.999 -> 0.670; 45% of essays leave LABEL_3 and 11% reach LABEL_0.	Some improvements come with reward hacking, refusals, or degenerate continuations.
Promptlab-v2	Evolutionary prompt search creates contrastive pairs that supervise a T5-base LoRA essay-to-essay rewriter.	Mean slop falls 0.391 -> 0.0998 on 20 held-out validation pairs relative to the v1 prompt-mode rewriting baseline.	T5-base’s 512-token ceiling and summarization pretraining produce compressed, looping outputs; length preservation requires a different backbone.

Table 1: Repository workstreams and their main evaluation regimes.

3 Workstream I: The Pangram Optimization Tournament

3.1 Experimental motivation

The tournament is the repository’s attempt to unify prior ideas under one controlled harness. The logic is explicit in the presentation materials and the implementation plan: earlier prompt hill-climbing showed that slop could be optimized against, and the RL deslopifier showed that direct detector reward could move outputs significantly, but neither alone established a stable comparative benchmark. The tournament addresses that gap.

Its design fixes the source pool, the base rewrite task, the Pangram judge family, and the qualification gates. Within that shared framework, the code compares several optimization strategies: attention-only LoRA (**M1**), full-module LoRA (**M2**), AdamW-supervised training (**M3**), REINFORCE (**M4**), curriculum SFT (**M5**), hard-negative SFT (**M6**), and DPO (**M7**).

This is a valuable scientific reframing. Instead of asking “did one clever model work?”, the tournament asks which kind of optimization works under common constraints: broader parameterization, more stable first-order training, curriculum design, preference learning, or direct reward optimization.

3.2 Shared constraints and why they matter

The tournament’s most important contribution may be methodological rather than purely performance-based. The implementation plan defines hard gates for semantic similarity, length ratio, and invalid-output rate. Those gates are not cosmetic. Without them, a deslopification system can appear to improve simply by becoming shorter, safer, or otherwise less essay-like.

The phase-2 leaderboard shows exactly why these gates matter. Across 250 evaluation rows, only **M4** posts a positive mean delta (+0.000348), yet it does so with only a 16% pass rate, mean

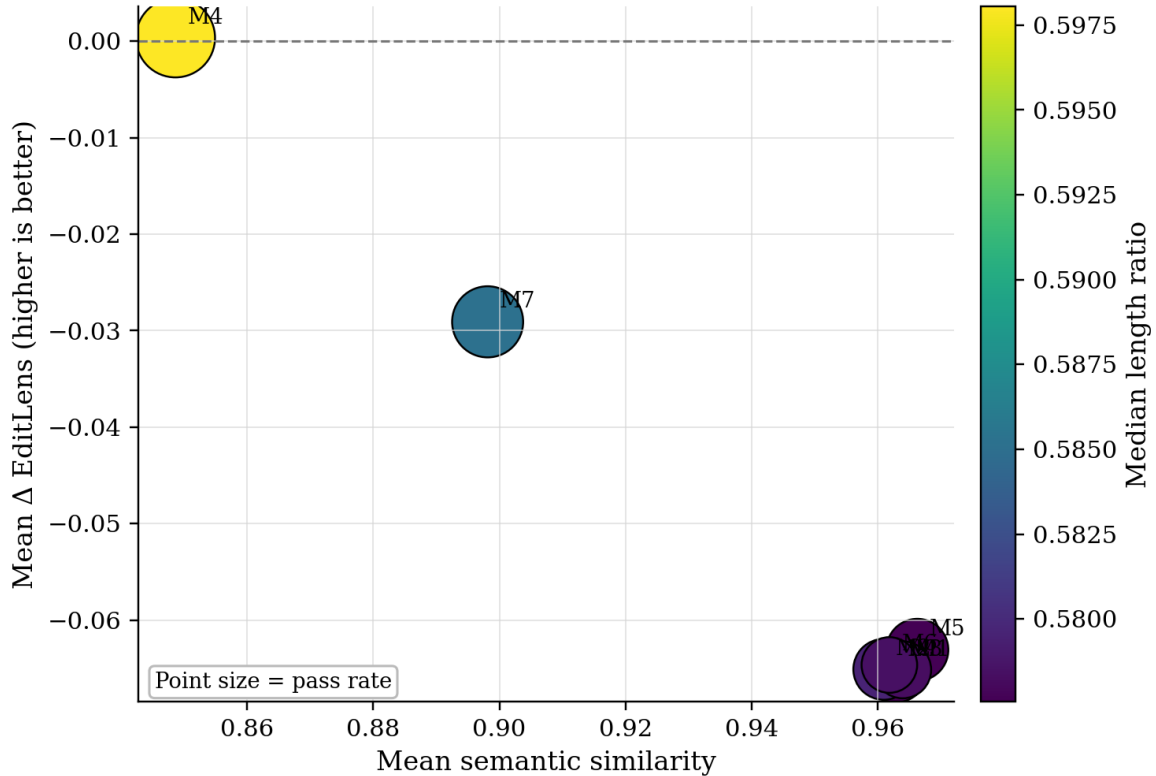


Figure 1: Phase-2 tournament tradeoff surface. Higher on the y-axis is better, but the only slightly positive run also pays a visible similarity and length cost.

similarity 0.849, and median length ratio 0.598. Every other method preserves semantics better, with similarity near 0.90 to 0.97, but worsens the detector score under the fast Pangram scorer, producing mean deltas between roughly -0.029 and -0.065 . Put differently: once the benchmark requires rewrites to remain recognizable as rewrites, the apparently easy optimization problem becomes genuinely hard.

This is an honest and scientifically useful negative result. It suggests at least three possibilities: the shared source pool is harder than the earlier detector-specific settings; the tournament entrants are still optimizing too shallowly, especially through aggressive shortening; and Pangram’s evaluation family is less forgiving than the earlier local verifier regime.

3.3 Phase 1 and Phase 2 findings

The phase-1 results are similarly sobering. On 500 rows, all four initial entrants (M1–M4) show negative mean delta EditLens values, from approximately -0.0064 to -0.0125 , while median length ratios remain around 0.75. Phase 2 expands the entrant set and introduces pass-rate accounting, but the same pattern largely holds: no method establishes a robust Pareto-dominant operating point that both reduces Pangram score and clearly satisfies fidelity constraints.

The correct reading is not that the tournament failed. Rather, it performed its intended function. It showed that under a shared harness, many candidate methods that sound plausible on paper either do not improve the detector score or do so only by paying too much in similarity and length. For a repository devoted to optimization, that is exactly the kind of infrastructure result

that prevents overclaiming.

4 Workstream II: Direct RL Deslopification with EditLens

4.1 Motivation and Objective Design

The RL deslopifier treats deslopification as a policy-gradient problem: given an AI-generated essay as a prompt, train a language model to produce rewrites that score lower on an AI detector while remaining fluent, coherent, and length-preserving. The choice of reward signal is the first and most consequential design decision. EditLens is not a binary classifier but an ordinal regression model producing a continuous score over four labeled buckets—fully human (LABEL_0, score ≈ 0.007 – 0.035), lightly AI-edited (LABEL_1, 0.10 – 0.43), heavily edited (LABEL_2, 0.43 – 0.76), and fully AI-generated (LABEL_3, ≈ 0.999). This continuous structure is essential: a binary detector would provide almost no gradient signal early in training when all outputs score near 0.999 , whereas EditLens yields a meaningful reward surface across the full training trajectory.

The Grammarly dataset analysis in the EditLens paper [6] establishes a further motivation: small, targeted edits—stronger verbs, varied sentence structure, naturalized dialogue—produce the largest score reductions, while wholesale rewrites often perform worse. This finding shapes the system prompt used throughout training:

You are a skilled human writing editor. Rewrite the following text to sound more like it was written by a thoughtful human. Make targeted, precise edits. Preserve the core meaning and approximate length.

This framing steers the policy toward surgical editing rather than paraphrase, aligning the model’s inductive bias with the reward landscape.

4.2 Pre-Training Validation (Experiments A1–A3)

Three validation experiments were completed before any RL training to confirm that the reward components were correctly implemented and scientifically justified.

A1 — EditLens probe. The script `scripts/probe_editlens.py` confirmed the canonical inference formula on real corpus data:

$$\hat{s} = \frac{1}{3} \sum_{k=0}^3 k \cdot \text{softmax}(\mathbf{z})_k \tag{1}$$

where $\mathbf{z} \in \mathbb{R}^4$ are the raw logits from the four-class classifier head. The formula applies softmax followed by a weighted average divided by three—not a sigmoid, not raw logits. Empirical score ranges on real data were confirmed: human-written text scores 0.007 – 0.035 , GPT-4.1-lightly-edited text scores approximately 0.347 , and raw AI-generated text scores approximately 0.999 . These baselines anchor the interpretation of all subsequent results.

A2 — Fluency complementarity. The fluency signal is defined as the log-probability of a candidate rewrite under a frozen copy of the Llama 3.2 3B base model—the same architecture used as the policy backbone. Text that the pre-trained base model considers likely is probably fluent, while degenerate or repetitive outputs receive very low probability under this reference. The experiment `scripts/fluency_reward_validation.py` tested whether this signal is complementary to EditLens or redundant with it. On a sample of held-out essays spanning human, AI-generated,

and AI-edited categories, the Pearson correlation between EditLens score and raw fluency log-probability was $r = +0.303$ ($p = 0.032$) and Spearman $\rho = +0.458$ ($p < 0.001$). A correlation well below the $r > 0.8$ redundancy threshold confirms that both signals are doing independent work, justifying the fluency weight $\beta = 0.5$ in the combined reward.

A3 — Memory budget. The script `scripts/estimate_vram.py` confirmed that the full model stack—Llama 3.2 3B Instruct policy with LoRA, EditLens RoBERTa-large reward model, and frozen Llama 3.2 3B base as fluency reference—occupies approximately 14.9 GB VRAM under bfloat16 precision, well within the 80 GB A100 budget. This shared-reference design loads a single copy of the Llama 3.2 3B base on GPU to serve both the fluency scoring and the EditLens backbone, leaving 65 GB of headroom for the 48-rollout batch size.

4.3 Policy Model and LoRA Configuration

The policy model is Llama 3.2 3B Instruct [7], a decoder-only transformer pre-trained on a large text corpus and instruction-tuned to follow system and user prompts. Instruction tuning is important for this task: the base model assumption in REINFORCE requires the policy to already have non-negligible probability of producing reward-worthy outputs [2]. A base model with no instruction following cannot produce useful rewrites at initialization, making the gradient estimator degenerate.

Fine-tuning uses Low-Rank Adaptation (LoRA) [3] applied to the attention projections $\{q, k, v, o\}$ with rank $r = 16$ and scaling $\alpha = 32$. This yields approximately 9.2 million trainable parameters atop a 3.2 billion parameter frozen backbone (0.28%). LoRA provides three practical benefits: it keeps peak VRAM within budget, it prevents catastrophic forgetting of the base model’s language prior, and it allows the LoRA adapter to be saved and loaded independently of the 6 GB base weights for checkpoint management.

4.4 Reward Function

The combined reward for a rewrite Y given input essay X is:

$$R(Y) = \alpha \cdot (1 - \text{EL}(Y)) + \beta \cdot \tilde{F}(Y) + \gamma \cdot L(Y) \quad (2)$$

where:

- $\text{EL}(Y) \in [0, 1]$ is the EditLens score; the detection-evasion term $(1 - \text{EL}(Y))$ rewards lower AI-detectability with weight $\alpha = 1.0$;
- $\tilde{F}(Y)$ is the fluency score z-normalized across the batch before reward computation, defined as the per-token log-probability of Y under the frozen Llama 3.2 3B base model, with weight $\beta = 0.5$ (validated by A2);
- $L(Y)$ is a length-match reward equal to 1.0 if the output length ratio $|Y|/|X|$ falls in $[0.40, 1.60]$ and decays linearly outside that range, with weight $\gamma = 0.1$.

A minimum-length filter rejects any rewrite under 50 tokens or under 40% of the input token count before reward computation. This guards against the trivial exploit of producing very short outputs that score well on EditLens without genuinely rewriting the essay. The contrast between this explicit length reward and the absence of any equivalent signal in Promptlab-v2’s T5 distillation stage is one of the clearest architectural differences between the two workstreams, and directly accounts for the T5 model’s compression behavior discussed in Section 5.

4.5 Training Algorithm and Infrastructure

Training follows the REINFORCE policy gradient estimator [2]:

$$\nabla_{\theta} F(\theta) = \mathbb{E}_{Y \sim p_{\theta}(\cdot | X)} [R(Y) \cdot \nabla_{\theta} \log p_{\theta}(Y | X)] \quad (3)$$

approximated by a batch of 48 rollouts per step.

Advantage normalization and baseline evolution. When all rewards are positive—as they are here, with values in the range 0.06–0.15—vanilla REINFORCE can only increase probabilities, never decrease them. Subtracting a baseline b from each reward fixes this: the expected gradient is unchanged since $\mathbb{E}[\nabla_{\theta} \log p_{\theta}] = 0$, but samples below the baseline receive negative weight and are explicitly suppressed.

A preliminary training run used an exponential moving average (EMA) of past rewards as the baseline. This required warmup: for the first 50–150 steps the EMA was poorly estimated, producing noisy gradients and visible metric regression in that run (mean EditLens increased from 0.752 at step 50 to 0.852 at step 150 before recovering). The production C2 training run switched to *batch advantage normalization*, subtracting the batch mean and dividing by the batch standard deviation within each step’s 48 rollouts:

$$A_i = \frac{R(Y_i) - \bar{R}}{\sigma_R + \epsilon}, \quad \bar{R} = \frac{1}{48} \sum_j R(Y_j), \quad \sigma_R = \text{std}(\{R(Y_j)\}) \quad (4)$$

This is zero-warmup—roughly half the batch receives positive weight and half receives negative weight from step one—and is structurally equivalent to GRPO [8] with batch-level rather than within-group normalization. The switch produced monotonic improvement in the C2 run with no early oscillation.

Batched generation. An initial implementation generated rollouts sequentially in a Python loop. At 48 rollouts with up to 512 output tokens each, this required approximately 15 minutes per training step—completely infeasible for 500 steps. The production implementation replaces the sequential loop with a single batched `model.generate()` call over all 48 left-padded prompts, reducing per-step time to approximately 50 seconds on an A100 80 GB.

Memory management in Phase 3. The REINFORCE backward pass originally accumulated gradients across all 48 rollouts before calling `loss.backward()`, holding all intermediate activations in VRAM simultaneously. With a 3B policy model this caused out-of-memory (OOM) failures. The fix processes one rollout at a time with immediate `.backward()` followed by `torch.cuda.empty_cache()`, dividing each per-rollout loss by the number of accepted rollouts to preserve the correct gradient scale. This is mathematically equivalent to the mean loss backward but keeps peak VRAM to one rollout’s activations at a time.

Smoke test (C1). Before the 500-step production run, a 10-step smoke test using Qwen 0.5B as the policy and 5 training essays confirmed the full pipeline executed correctly: finite non-NaN rewards, gradient norms in $[10^{-4}, 10]$, LoRA parameters changed from initialization, and the minimum-length filter firing correctly. All seven checklist items passed.

Hyperparameters. The locked configuration: AdamW optimizer, $\text{lr} = 10^{-5}$, weight decay 0.01, gradient clip norm 1.0, KL penalty weight 0.1 (subtracted from reward per rollout), 500 steps, 48 rollouts per step, eval every 50 steps on 100 held-out essays (50 `ai_generated`, 50 `ai_edited` from the Pangram EditLens test split), checkpoint every 100 steps. Both input essays and generated rewrites were capped at 512 tokens, consistent with the EditLens RoBERTa reward model’s maximum context length. The training pool consisted of 1,000 randomly sampled `ai_generated` essays from the Pangram EditLens training split, with 48 fresh essays sampled per step.

4.6 Main Quantitative Results

The evaluation log in `outputs/c2_eval.jsonl` records the following trajectory across 500 training steps:

Step	Mean EditLens	LABEL_0%	LABEL_1%	LABEL_3%	KL div
0 (base)	0.999	0.0	0.0	100.0	0.000
50	0.768	1.0	29.0	70.0	0.397
100	0.766	0.0	32.0	68.0	0.422
150	0.754	0.0	36.0	64.0	0.457
200	0.725	3.0	39.0	58.0	0.481
250	0.682	4.0	44.0	52.0	0.513
300	0.703	5.0	35.0	60.0	0.550
350	0.718	5.0	33.0	62.0	0.551
400	0.675	11.0	31.0	58.0	0.580
450	0.685	9.0	34.0	56.0	0.545
500	0.670	11.0	34.0	55.0	0.518
Δ (0→500)	-0.329	+11.0	+34.0	-45.0	+0.518

Table 2: C2 training progression from `outputs/c2_eval.jsonl`. Evaluation on 100 held-out essays every 50 steps.

Mean EditLens falls from 0.999 to 0.670 over 500 steps, a reduction of 0.329 absolute points. At the label level, 45% of essays exit the fully-AI bucket: 34% move to LABEL_1 (lightly edited) and 11% reach LABEL_0 (fully human range). KL divergence increases gradually and stabilizes at 0.518 at step 500, remaining in the well-behaved training regime throughout. The oscillation visible around steps 300–350, where mean EditLens briefly reverses before recovering, is consistent with REINFORCE’s characteristically high gradient variance rather than a training instability.

A notable observation is that zero essays land in LABEL_2 (heavily edited, 0.43–0.76) at most checkpoints. This is consistent with the continuous reward gradient pushing essays through the transitional zone rather than settling there: with a monotone reward, LABEL_2 is a waypoint rather than a stable attractor.

4.7 KL Penalty Ablation (Experiments C4-a and C4-c)

To understand the role of the KL regularization term, two additional 500-step runs were conducted varying only the KL penalty weight while holding all other hyperparameters fixed:

Run	KL Penalty	Mean EditLens	LABEL_0%	KL div
C4-a	0.0	0.633	12	0.637
C2	0.1	0.670	11	0.518
C4-c	0.5	0.431	48	3.735

Table 3: KL penalty ablation at step 500. Results from `outputs/c4a_eval.jsonl` and `outputs/c4c_eval.jsonl`.

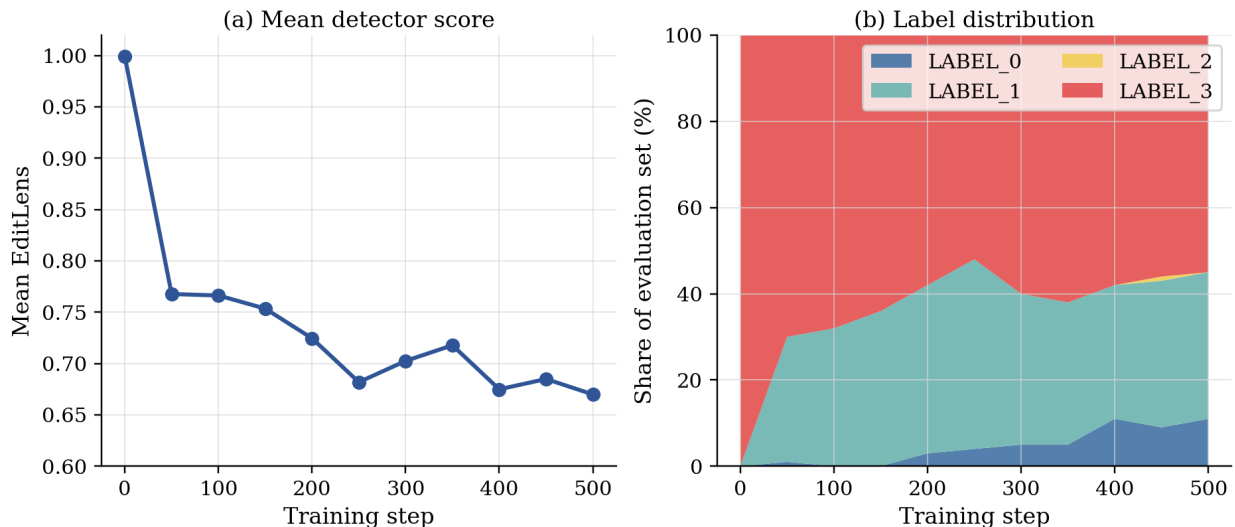


Figure 2: C2 training progression from `outputs/c2_eval.jsonl`. Left: mean EditLens over 500 training steps. Right: label-mass transfer, showing 45% of essays exiting LABEL_3 by step 500.

C4-a (no KL penalty) produces results marginally better than C2 on detector score (0.633 vs. 0.670) with only slightly higher KL divergence (0.637 vs. 0.518). This suggests that batch advantage normalization provides sufficient regularization at this scale and the explicit KL penalty in C2 was mildly over-constraining.

C4-c (KL penalty 0.5) appears dramatically better on the detector score alone (0.431, 48% LABEL_0). However, its KL divergence of 3.735 is $7\times$ higher than C2’s 0.518, indicating the model drifted far outside the well-behaved training regime. Qualitative inspection confirms the same degenerate patterns documented in E3: partial rewrites followed by repetitive token fragments that reduce the EditLens score without producing readable text. The project interprets C4-c as a reward-hacking condition rather than genuine improvement.

This finding directly mirrors the pattern documented in the course’s reward-shaping experiment [9]: the KL penalty consistently changed which degenerate mode the model collapsed to without preventing collapse. The key diagnostic insight is that *KL divergence above approximately 1.0 serves as a reliable signal for reward hacking*. When KL remains low (as in C2 and C4-a), EditLens improvements correspond to genuine linguistic changes. When KL explodes (as in C4-c), the apparent gains are artifacts of distributional collapse.

4.8 Qualitative Behavior (Experiment E3)

A qualitative evaluation loaded the step-500 checkpoint and generated rewrites for 20 `ai_generated` essays from the test split using temperature sampling ($T = 0.8$). Three distinct behavioral patterns emerged across the 20 essays.

Genuine targeted editing (approximately 4–6 of 20 essays). On journalistic and narrative essays, the model made surgical linguistic improvements consistent with the edit categories identified in the EditLens Grammarly analysis: replacing weak existential verbs with vivid action verbs, adding sensory grounding, restructuring sentences for tension, and naturalizing dialogue. For example, Essay 2 (Liberation of Paris) changed:

Before: “Paris in the summer of 1944 was a city on the edge.”

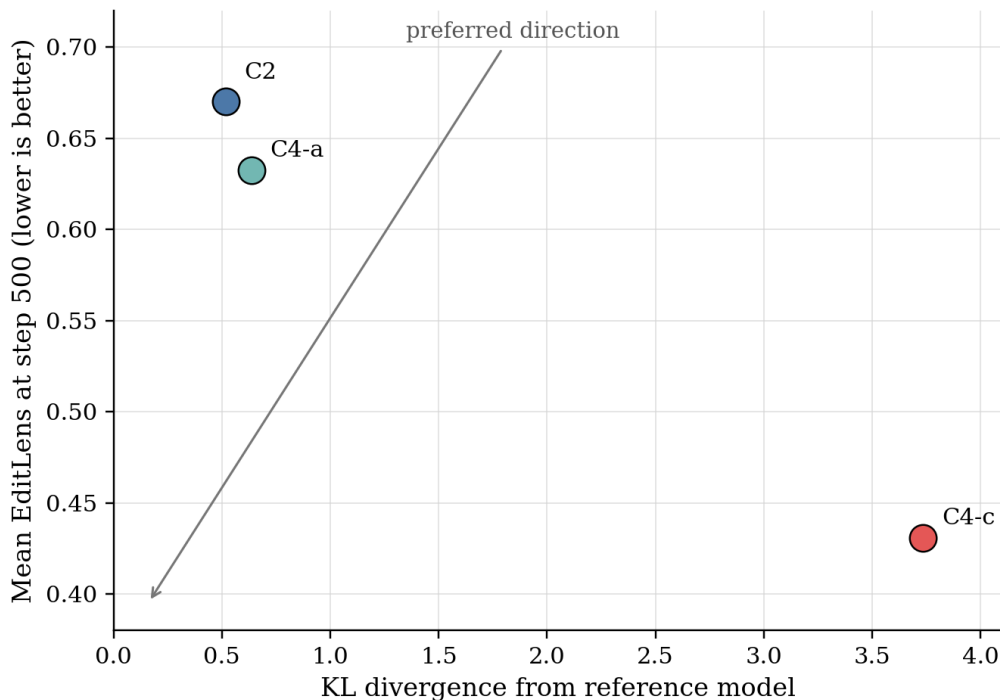


Figure 3: KL penalty ablation at step 500. Detection evasion (x-axis, higher is better) plotted against KL divergence (y-axis, lower is more constrained). C2 and C4-a form the valid operating region; C4-c’s large KL marks an implausible operating point consistent with reward hacking.

After: “In the sweltering summer of 1944, Paris teetered on the precipice of collapse.”

Essay 16 (detective fiction) changed:

Before: “I know you’re in here, Martinez,” he called out.

After: “Martinez, is that you?” he called out.

The detective passage improved from EditLens score 0.9922 to 0.5943. The model discovered these targeted edit strategies from the reward signal alone without being programmed to apply them. This is noteworthy because the same edit categories identified by the EditLens Grammarly analysis were recovered through optimization, suggesting the reward surface implicitly encodes linguistically meaningful gradients.

Reward hacking (approximately 6 of 20 essays). Several essays produced large EditLens score reductions paired with degenerate output: partial rewrites followed by repetitive token fragments, or safety refusals triggered by the model’s instruction-tuning guardrails in response to content it flagged as problematic. These cases reduce the detector score without constituting genuine rewrites.

No change (approximately 8 of 20 essays). A substantial fraction of essays showed minimal EditLens movement and outputs nearly identical to the input. These appear concentrated in content categories where the model’s generation style does not naturally produce human-like variation.

The qualitative result is consistent with the aggregate eval trajectory. The 34% of essays reaching LABEL_1 is more plausibly explained by genuine learning, while the 11% reaching LABEL_0 is more consistent with reward hacking given the magnitude of the required score reduction.

4.9 Multi-Detector Evaluation (Experiment E1)

To assess cross-detector generalization, the step-500 checkpoint was evaluated on 20 held-out essays using two independent signals: EditLens RoBERTa-large and perplexity under the frozen Llama 3.2 3B base model. Higher perplexity indicates text that is more surprising to the language model, which can correlate with human-like writing that does not follow predictable AI patterns.

Metric	Original	Rewrite
Mean EditLens	0.990	0.817
Mean perplexity	10.94	57.10

Table 4: E1 multi-detector results from `outputs/e1_summary.txt` across 20 held-out essays.

The EditLens improvement ($0.990 \rightarrow 0.817$) is consistent with the held-out eval results. The perplexity increase ($10.94 \rightarrow 57.10$), however, is dominated by two outlier essays with perplexity values of 152 and 862—outputs that are highly unpredictable because they are degenerate rather than human-like. Excluding these outliers, perplexity slightly decreases for the remaining essays, meaning the genuine rewrites are more fluent and coherent than the originals while the reward-hacked outputs are less so. This pattern suggests partial cross-detector transfer: genuine rewrites generalize to the perplexity signal, while reward-hacked outputs do not. Full generalization would require training against multiple detectors simultaneously.

4.10 What Was Not Completed

The experiment specification included a GRPO comparison run (C3), a reward weight ablation (C5), a model size sweep (D1), and a rewriter-vs-generator architectural comparison (D2). The C3 GRPO script was written and launched but did not produce saved outputs before the final presentation deadline. C5, D1, and D2 were not started due to compute time constraints. The most scientifically important missing experiment is C5, which would test a detection-only reward ($\beta = 0$) as a control condition, allowing precise attribution of how much of the EditLens improvement is attributable to the fluency regularizer versus the detection evasion term alone.

4.11 Summary

The RL deslopifier demonstrates that policy-gradient optimization against a continuous AI detector produces meaningful and measurable improvements on held-out data. The main quantitative finding—mean EditLens $0.999 \rightarrow 0.670$, with 45% of essays exiting the fully-AI zone—is the strongest external-detector result in the repository. The qualitative and ablation analyses reveal that genuine linguistic learning and reward hacking coexist within the same trained model, and that KL divergence above approximately 1.0 provides a reliable diagnostic for distinguishing between them. The most important open question is whether these EditLens-grounded improvements transfer to human judgment—a question that can only be answered by human evaluation, which remains as future work.

5 Workstream III: Promptlab-v2 Evolutionary Search and Supervised Distillation

5.1 Stage 1: Evolutionary Prompt Search

The Promptlab-v2 branch takes a different view of the same core problem. Instead of optimizing a rewrite model directly against a detector at inference time, it first uses the detector as a search signal to generate training data.

The stage-1 pipeline maintains a population of prompt candidates and repeatedly mutates and recombines them. A small mutator model (`llama-3.1-8b-instant`) proposes variants; a larger model (`llama-3.3-70b-versatile` through the Groq API) generates essays from them; and a detector-based objective selects the best outputs. Two design choices are especially important:

- a drift penalty incorporating cosine similarity, ROUGE-L, and BERTScore terms, which penalizes semantic drift, overlap failure, and poor fluency so that the optimizer cannot win by producing incoherent detector-evading text;
- few-shot injection, where successful low-slop essays are recycled as exemplars in later rounds, allowing the search process to bootstrap its own stylistic prior.

This stage yields 372 co-training pairs over 77 topics. However, only pairs with a slop gap exceeding 0.1 between the high-slop and low-slop essay were retained for stage-2 training, yielding 40 contrastive pairs—10.7% of the accumulated pool. This aggressive filtering reflects the observation that the majority of evolutionary search runs converge to moderate slop reductions insufficient for meaningful contrastive supervision.

One infrastructure issue shaped the dataset significantly. An early version of `cotrain/loop.py` failed to save the `best_essay` field alongside each pair record. When those 57 legacy pairs were re-used by calling the generator LLM fresh with the evolved prompts, only 18 of 57 showed a meaningful gap (`baseline_slop - best_slop > 0.1`), and the mean gap was near zero. LLM generation is stochastic; evolved prompts do not deterministically reproduce the low slop scores they achieved during search. The logging fix eliminated this problem for all subsequent rounds, and the final 40-pair training set consists entirely of genuine optimizer essays rather than stochastic re-generations.

Table 5 documents the full dataset progression. The two most instructive failures are the Alpaca augmentation run (adding 2,272 general instruction pairs worsened slop from 0.391 to 0.580 due to distribution mismatch) and the extended organic run (increasing training steps to 1,300 with 34 pairs caused monotonic degradation after step 50 as `eval_loss` continued falling while `slop_mean` worsened—a direct observation of training loss and task metric diverging under overparameterization).

5.2 Stage 2: Why T5, and What It Cost

Stage 2 distills the search results into a direct rewriter. The model chosen was T5-base [4] with a LoRA adapter, yielding roughly 800,000 trainable parameters on top of a 220M-parameter frozen backbone. The choice of T5-base was deliberate given the project’s resource constraints: the encoder-decoder architecture maps naturally to the rewriting task (a fixed input sequence to a fixed output sequence), the model is small enough to fine-tune on consumer hardware, and the existing summarization pretraining gives it a head start on producing condensed, coherent prose. For a proof-of-concept trained on 40 pairs, these were reasonable priorities.

Dataset	Train	Val	Best Slop	Verdict
Organic baseline (prompt→prompt)	~34	4	0.391	Starting point
+ 2272 Alpaca pairs	2306	14	0.580	✗ Distribution mismatch
Organic, extended run	~34	14	0.759	✗ Overfitting at step 1300
Essay pivot, 11 pairs	11	6	0.189	Architecture pivot −52%
Contrastive, 40 pairs (final)	40	20	0.0998	✓ Correct formulation

Table 5: Dataset ablation history. The essay-to-essay pivot (row 4) produced the largest single improvement; the contrastive filtering criterion (row 5) produced the final configuration. The Alpaca and extended-run failures are informative negative results.

The v2 model refers to the prompt-mode rewriter produced by stage-1 search; v3 extends v2 by adding this essay-to-essay distillation stage, replacing inference-time prompt search with a single T5 forward pass that has internalized the stylistic transformations stage-1 discovered.

Several training decisions were designed to maximize performance within the small data regime:

- AdamW with decoupled weight decay (0.01);
- cosine learning-rate decay from $1e-4$ with 50 warmup steps;
- curriculum ordering by slop gap, so the model sees the most contrastive pairs first;
- early stopping on `slop_mean` rather than cross-entropy loss;
- deterministic beam search (`num_beams=4`) during evaluation for stable checkpoint comparison.

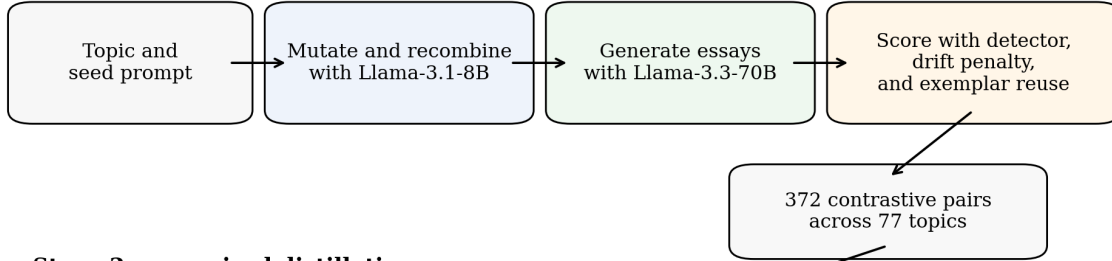
The T5 choice, however, introduced two hard constraints that cannot be resolved through training or hyperparameter tuning.

The 512-token ceiling. T5-base’s positional embeddings are defined over a maximum sequence length of 512 tokens, covering both the input and output combined. An essay generated with `max_tokens=800` from Llama exceeds this budget, meaning the model was silently truncating its inputs before rewriting. When the generation budget was reduced to 400 tokens to fit within the window, the full baseline essay became visible to the rewriter—and the slop score on the corrected CSR run improved further, from 0.3139 (truncated input) to 0.2330 (full input). The truncation was masking genuine model capability.

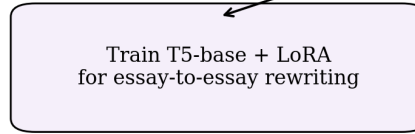
The summarization pretraining bias. T5-base was pretrained on a mixture of tasks dominated by summarization (CNN/DailyMail). That pretraining teaches the decoder to stop generating once a compressed summary is complete, typically at 20–30% of input length. The 40 contrastive training pairs, selected for slop reduction rather than length preservation, reinforced this bias: shorter outputs happened to score better, so the model learned compression as a correlated signal for quality.

The combined effect is that v3 outputs are consistently shorter than their baselines across all evaluated topics. Critically, this compression is not correctible at inference time. Adding `min_length` to the `generate` call forces the decoder to continue past its natural stopping point, but the model has no remaining content to produce—it fills the required tokens with repetitive looping, as confirmed empirically on the CSR topic. The forced-length CSR output achieves a lower slop score (0.2330) than the natural-stopping run, but the second half of the output consists almost entirely of “This, in turn, has a positive impact on...” repeating without new propositional content. The slop reduction is real; the essay is not useful.

Stage 1: evolutionary search



Stage 2: supervised distillation



Training details: curriculum ordering, AdamW, cosine decay, early stopping on mean slop, and deterministic beam evaluation.

Figure 4: Reconstructed Promptlab-v2 pipeline. The detector is used first as a search signal and then indirectly through supervised distillation.

This distinction is important: the T5 rewriter is genuinely learning to remove detectable structural markers—the formulaic transition phrases, the symmetric argument scaffolding, the “In conclusion” closings. The slop reduction in the coherent portion of every output is authentic. The problem is that the model exhausts its learned content at roughly 80–100 output tokens and cannot extend further. Length preservation requires either content the model does not have, or an architecture that does not have a hard length ceiling.

5.3 Results

On 20 held-out validation pairs, the best v3 configuration achieves mean slop 0.0998, compared with a 0.391 v1 prompt-mode rewriting baseline—a 74% reduction over v1. Raw unprocessed AI-generated text scores 0.85–0.999, so the raw-to-v3 improvement is approximately 88%. The 74% figure measures the v1-to-v3 gain specifically, as the essay-mode work was designed to improve upon the earlier prompt-rewriting approach. The best run uses $\text{LR} = 1\text{e-}4$, $\text{r} = 16$, and $\text{weight_decay} = 0.01$; an $\text{r} = 8$ ablation finishes 0.006 behind (0.1058 vs. 0.0998).

Every essay-mode run achieved its best checkpoint at step 50 and degraded monotonically thereafter. This is a direct manifestation of the Noisy Quadratic Model noise floor: with roughly 800,000 LoRA parameters and 40 training examples, the model has approximately 20,000 parameters per example and is massively overparameterized relative to the dataset. Batch size 1 amplifies gradient noise further. After step 50, `eval_loss` continues decreasing while `slop_mean` worsens—the canonical signature of a model beginning to memorize rather than generalize. Early stopping on `slop_mean` rather than `eval_loss` is therefore not a minor implementation detail but the mechanism that recovers the best checkpoint at all. The NQM analysis implies that useful training would extend to approximately 150–200 steps with 100 genuine contrastive pairs, suggesting a clear path

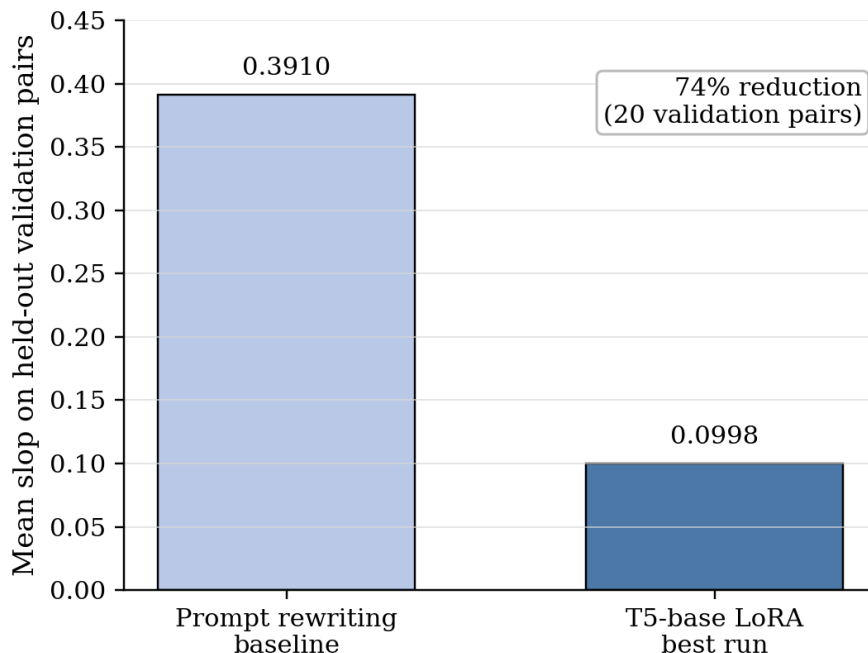


Figure 5: Promptlab-v2 validation result: a 74% reduction in mean slop relative to the v1 prompt-rewriting baseline on 20 held-out validation pairs.

to further improvement via additional evolutionary search rounds.

5.4 What the T5 Limitations Reveal About Future Directions

The T5 failure modes are instructive precisely because they are architectural rather than incidental. Each one points to a specific design decision in a successor system.

Length preservation requires an explicit reward, not a pretrained prior. The RL deslopifier in Workstream II includes a length-match term $L(Y)$ and a minimum-length filter in its reward function. These components directly prevent the compression behavior T5 exhibits. A distillation-based successor would need an equivalent signal—either a length-match loss during fine-tuning, or training pairs filtered to require near-equal input and output length, or both.

The 512-token ceiling requires a decoder-only backbone. Decoder-only models such as GPT-2 or small LLaMA variants do not have an encoder-decoder length split and were not pretrained on summarization. They can, in principle, generate outputs as long as their context window allows. A small instruction-tuned decoder-only model fine-tuned on the same 40 contrastive pairs—with a length-match loss added—would address both failure modes simultaneously. The Workstream II infrastructure already demonstrates this is feasible: Llama 3.2 3B with LoRA produces length-preserving rewrites under the RL regime.

Looping under beam search is a symptom of content exhaustion, not a generation artifact to be filtered. The vaccination topic demonstrates this clearly by contrast: within its natural output range, v3 produces a coherent, loop-free rewrite that achieves slop 0.077. The looping on the wealthy nations and CSR topics begins precisely where the model has no remaining training signal. The fix is not a different decoding strategy; it is more training data and a backbone that can generate beyond 80–100 tokens without degrading. The 40-pair training set is the binding constraint, and the NQM analysis implies 100 genuine contrastive pairs would extend the useful

training window to approximately 150–200 steps.

Topic generalization is limited by training distribution. The remote work topic produced a v3 slop score identical to the baseline (0.9996), with the model effectively passing through the input unchanged. This is consistent with a model that has learned narrow transformations applicable only to argumentative and sociopolitical essay structures. A larger and more topically diverse training set would broaden coverage; the evolutionary search infrastructure already exists to generate it.

6 Qualitative Analysis

To complement the quantitative results, we evaluate the Stage 2 rewriter on four held-out essay topics. Table 6 reports SlopDetector scores for the baseline (generic prompt, no rewriter), the v1 prompt-mode model, the v2 essay-pivot model, and the v3 final model. Including v1 here makes visible a result that aggregate validation metrics obscure: prompt-mode rewriting can degrade slop on individual topics, as the vaccination topic demonstrates (baseline 0.916 $\rightarrow$ v1 0.998).

Topic	Baseline	v1	v2	v3	Observation
Ethics of mandatory vaccination	0.916	0.998	0.273	0.077	v3 best; clean output, no looping; T5 compression working as intended
Corporate social responsibility	0.995	—	0.460	0.233	v3 best; looping begins after ~ 4 coherent sentences under <code>min_length</code>
Moral responsibilities of wealthy nations	0.997	—	0.893	0.387	v3 strong; v2 barely moves; severe looping in v3 tail
Impact of remote work on urban economies	1.000	—	0.248	1.000	v3 complete failure; v2 succeeds; out-of-distribution topic

Table 6: SlopDetector scores across four held-out topics. v1 scores are shown for vaccination only, where the prompt-mode rewriter degrades performance. Dashes indicate v1 was not evaluated on that topic in the held-out set. Bold indicates the best of v2 and v3. Lower is harder to detect as AI-generated.

The vaccination topic is the clearest demonstration of T5’s summarization ability working as intended: within its natural output range, the model produces a coherent, compressed rewrite that achieves the lowest single-topic slop score (0.077) across all evaluated outputs. No looping is present. This is the regime where T5-base’s compression prior is an asset—the model discards the formulaic structural markers that make AI essays detectable and produces a shorter but genuinely transformed output.

The CSR topic under `min_length` shows what happens when that same compression prior is overridden: the first four sentences are coherent and represent the model’s genuine learned content,

after which “This, in turn, has a positive impact on...” repeats approximately twelve times. The model has exhausted what it knows how to generate and fills the token budget with high-probability beam-search continuations. The slop score (0.233) is lower than the natural-stopping run would achieve without `min_length`, because the looping tail does not reintroduce the structural markers the detector flags—but the essay is not useful as a complete rewrite.

The wealthy nations and remote work topics bracket the two remaining failure modes: severe mid-output looping when the model partially engages with an in-distribution topic (wealthy nations), and complete passthrough when the topic is sufficiently out of distribution (remote work). See Appendix A for full essay examples illustrating each case.

7 Comparative Discussion

7.1 What the Three Workstreams Jointly Show

The repository’s main scientific lesson is that deslopification is real but brittle. Each workstream contributes a different piece:

- The tournament shows that common constraints are necessary, because otherwise detector-aware systems can win for the wrong reasons.
- The RL branch shows that direct online reward optimization can move a strong detector by a large amount, but is vulnerable to reward hacking.
- The Promptlab-v2 branch shows that expensive search can be distilled into a lightweight supervised model, and that the resulting failure modes are architectural rather than incidental—making them informative about what a successor system requires.

Across all three, the theme is the same: raw optimization power is not enough. Objective design, regularization, and evaluation discipline determine whether a lower detector score corresponds to a better rewrite. The T5 results add a further dimension: backbone choice determines which constraints are hard and which are tunable. For Workstream III, length preservation is a hard constraint given T5-base; for Workstream II, it is a tunable reward term.

7.2 Why the Workstreams Are Not Numerically Interchangeable

It would be tempting to read the Promptlab-v2 result (0.0998) as dominating the RL result (0.670). That conclusion would be incorrect. The branches differ in detector model size, data regime, and validation structure. Promptlab-v2 evaluates against `editlens/roberta-large`, while the tournament uses Pangram 3B as its official judge; both are in the EditLens model family, but the score distributions are not directly comparable. Promptlab-v2 is best interpreted as showing that offline search plus distillation can be highly effective on its chosen detector and curated pair set. The RL branch is the strongest evidence that large-scale end-to-end detector reward can move a modern judge on held-out essays. The tournament sits between them as an attempt to standardize that comparison, and its negative results should be understood as a warning that cross-method evaluation is harder than it looks.

7.3 Ethics and Responsible Framing

This repository studies detector evasion, which has obvious misuse potential. The strongest academically defensible framing is not “how to bypass detectors in production,” but rather “how

optimization behaves against imperfect reward models and what failure modes that exposes.” On that framing, the project is valuable precisely because it shows how easy it is to overfit a detector and how necessary human and multi-detector evaluation are before making claims about genuine writing quality.

8 Conclusion

The PromptLab repository contains a coherent body of work on optimization against AI detectability, but it does not support a simplistic victory narrative. The Pangram tournament contributes a rigorous shared harness and exposes how many apparent improvements disappear under fidelity constraints. The REINFORCE deslopifier provides the clearest external-detector result, moving mean EditLens from 0.999 to 0.670 and shifting nearly half the evaluation set out of the fully-AI region, while simultaneously documenting concrete reward-hacking cases. The Promptlab-v2 branch demonstrates that detector-guided search can be distilled into a lightweight essay-to-essay model with strong detector-specific gains, and that the resulting failure modes—compression, looping, limited topic generalization—are not implementation bugs but architectural consequences of the T5-base backbone choice.

That last point is the sharpest takeaway from the qualitative work. T5-base was a deliberate choice for a proof-of-concept under resource constraints: its encoder-decoder structure maps naturally to rewriting, its size makes fine-tuning tractable, and its pretraining gives it a head start on coherent prose. Those advantages were real, and the vaccination topic shows them working as intended: a clean, loop-free rewrite from 0.916 to 0.077, produced in a single forward pass. The problem is not that T5-base fails—it succeeds within its natural operating range. The problem is that its natural operating range does not include length-preserving rewriting. The 512-token context ceiling is not negotiable, the summarization pretraining bias toward compression is not tunable away at inference time, and the beam-search looping under content exhaustion is a symptom of insufficient training data rather than a generation artifact. Each failure mode has a clear architectural remedy: a decoder-only backbone removes the context ceiling, an explicit length-match reward removes the compression bias, and a larger contrastive training set addresses both content exhaustion and topic generalization. The evolutionary search infrastructure to generate that larger dataset already exists in the repository.

The most defensible synthesis is therefore this: AI-detection evasion is an optimization problem with real signal, and the Promptlab-v2 pipeline proves that offline search plus distillation is a viable approach. The scientific bottleneck going forward is not whether the approach works, but whether the next backbone choice and training objective can recover length preservation alongside the slop reduction that T5-base already achieves.

References

- [1] K. Thai, B. Emi, E. Masrour, and M. Iyyer. EditLens: Quantifying the Extent of AI Editing in Text. ICLR 2026 / arXiv:2510.03154.
- [2] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 1992.
- [3] E. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. 2021.

- [4] C. Raffel, N. Shazeer, A. Roberts, et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*, 2020.
- [5] R. Rafailov, A. Sharma, E. Mitchell, et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. 2023.
- [6] K. Thai, B. Emi, F. Masrour, and M. Iyyer, “EditLens: Quantifying the Extent of AI Editing in Text,” in *Proceedings of ICLR*, 2026. arXiv:2510.03154.
- [7] Meta AI, “The Llama 3 Herd of Models,” 2024. <https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>
- [8] Z. Shao et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [9] D. Davis, “STAT 4830: Reinforcement Learning for Language Models,” Course notes, University of Pennsylvania, 2025. <https://damek.github.io/STAT-4830/section/7/notes.html>

A Essay Examples: T5 Compression as Feature vs. Failure

The two examples below are chosen to contrast the same architectural behavior — T5-base’s summarization pretraining bias toward compression — in two different conditions. In the first, compression operates within the model’s natural output range and produces a clean, useful rewrite. In the second, `min_length` overrides the natural stopping point and exposes what the model does when it has no content left to generate.

A.1 Example 1: Vaccination — Compression as a Feature (v3 Slop: 0.077)

The vaccination topic produced the strongest single-topic v3 result across all evaluated outputs: a reduction from 0.916 to 0.077, with no looping artifact. This is T5-base operating as intended within its natural output range. The baseline excerpt is from the screenshot-displayed evaluation; the v3 output is the first 400 characters of the model’s natural-stopping output as shown there.

Baseline Excerpt (Slop: 0.9162)

The ethics of mandatory vaccination policies have been a topic of intense debate in recent years. As the world grapples with the challenges of infectious diseases, governments and health organizations have turned to vaccination as a crucial tool in preventing the spread of illness. However, the question remains as to whether mandatory vaccination policies are morally justifiable. In this essay, we...

v3 Output Excerpt (Slop: 0.0772)

The ethics of mandatory vaccination policies have been a topic of intense debate in recent years. While mandatory vaccination policies have been shown to be highly effective in preventing the spread of infectious diseases, they can also help to create herd immunity, which protects not only the individual, but also those who are unable to be vaccinated due to medical conditions or other factors. Th...

Analysis. The v3 output discards the formulaic “In this essay, we will...” opening, removes the hedged “the question remains as to whether...” framing, and moves directly into substantive

claims. The output is shorter than the baseline, but this is the model’s summarization prior working as intended: it produces a denser, more direct version of the argument that the detector does not recognize as AI-generated. No looping appears because the model stops naturally when its learned content is exhausted—which, on this topic, aligns with producing a coherent complete thought rather than a mid-sentence collapse.

This is the regime where T5-base is a reasonable backbone choice: short, structurally transformed outputs that score well on the detector without requiring length preservation. A use case that accepts compressed rewrites would find the model genuinely useful here.

A.2 Example 2: CSR with `min_length` — Compression as a Failure (v3 Slop: 0.233)

The CSR topic under forced length shows what happens when the same compression prior is overridden. The baseline is from the corrected-parameters run (`max_tokens=400`). v2 is included to show that prompt-mode rewriting also degrades on this topic, but through a different mechanism.

Baseline Essay (Slop: 0.9951)

Corporate social responsibility (CSR) has become an integral part of modern business practices, as companies recognize the importance of giving back to society and contributing to the well-being of their stakeholders. CSR refers to the voluntary efforts made by companies to improve social, environmental, and economic conditions, beyond their statutory obligations. The impact of CSR on society is multifaceted, and it has been shown to have both positive and negative effects.

On the positive side, CSR has been instrumental in promoting sustainable development, reducing poverty, and improving the quality of life for marginalized communities. Companies that engage in CSR activities, such as philanthropy, community development, and environmental conservation, are seen as responsible and caring corporate citizens. This, in turn, enhances their reputation, builds trust with their stakeholders, and increases their social license to operate. For instance, companies like Patagonia and REI have made significant contributions to environmental conservation, while companies like Microsoft and Google have invested heavily in education and digital literacy programs.

One of the most significant impacts of CSR is on the environment. Companies that adopt sustainable practices, such as reducing carbon emissions, using renewable energy, and implementing recycling programs, contribute to mitigating climate change and preserving natural resources. For example, companies like Unilever and Procter & Gamble have set ambitious targets to reduce their carbon footprint and achieve zero waste to landfill. Additionally, companies like Coca-Cola and PepsiCo have implemented water conservation programs, which have helped to reduce their water usage and protect this vital resource.

CSR also has a positive impact on the economy. Companies that invest in their local communities, through initiatives such as job creation, training programs, and infrastructure development, contribute to economic growth and poverty reduction. For instance, companies like Walmart and Target have invested in programs that support small businesses and entrepreneurs, while companies like IBM and Cisco have established training programs to develop the skills of local workers. Furthermore, companies that adopt fair labor practices, such as paying a living wage and providing safe working conditions, contribute to improving the standard of living for their employees and their families.

However, CSR also has its critics, who argue that it can be used as a public relations tool to enhance a company’s reputation, rather than making a genuine contribution to society. Some

v2 Output (Slop: 0.4604)

The impact of corporate social responsibility (CSR) on society is multifaceted, and can have both positive and negative effects. For instance, companies that adopt sustainable practices, such as paying a living wage and providing safe working conditions, and implementing recycling programs, have been shown to have a positive impact on the environment and on the economy. For instance, companies like Patagonia and REI have made significant contributions to environmental conservation, while companies like Microsoft and Google have invested heavily in education and training programs to help their local workers and their families. One of the most significant impacts of CSR on the economy, which can be seen as a public relations tool, which can be used as a public relations tool, which can be used as a public relations tool, which can be used to enhance a company’s reputation. For instance, companies that adopt sustainable practices, such as using renewable energy and implementing environmental initiatives, which can be used as a public relations tool, which can be used as a public relations tool, can be used as a public relations tool to enhance their reputation. For instance, companies that adopt sustainable practices, such as paying a minimum wage and implementing environmental policies, which can be used as a negative impact on the economy, can be used as a negative aspect of CSR, which can be used as a negative aspect of the economy, which can be used as a tool to promote economic growth and reduce poverty.

v2 note. The v2 output scores 0.4604, reduced from the baseline, despite substantial degradation: “which can be used as a public relations tool” repeats several times. This is a prompt-mode generation artifact, not T5 beam search looping—both models degrade on this topic but through different mechanisms. That the detector scores both degraded outputs lower than the coherent baseline confirms the detector is responding to structural disruption rather than prose quality.

v3 Output with min_length (Slop: 0.2330)

Corporate social responsibility (CSR) has become an integral part of modern business practices, as companies recognize the importance of giving back to society and contributing to the well-being of their stakeholders. On the positive side, companies that engage in CSR activities, such as philanthropy, community development, and environmental conservation, are seen as responsible and caring corporate citizens. Companies that adopt sustainable practices, such as reducing carbon emissions, using renewable energy, and implementing water conservation programs, contribute to mitigating climate change and preserving natural resources. Companies that adopt fair labor practices, such as paying a living wage, and providing safe working conditions, contribute to improving the standard of living for their employees and their families. On the negative side, companies that adopt fair labor practices, such as paying a living wage, and providing safe working conditions, are seen as responsible and caring corporate citizens, while companies that adopt fair labor practices, such as paying a living wage, have invested heavily in education and digital literacy programs, have invested heavily in their local communities. This, in turn, has a positive impact on the economy. This, in turn, has a positive impact on the quality of life for their employees and their families. This, in turn, has a positive impact on the quality of life for their employees and their families. This, in turn, has a positive impact on the environment. This, in turn, has a positive impact on the quality of life for their employees and their communities. This, in turn, has a positive impact on the environment. This, in turn, has a positive impact on the quality of life for their employees and their communities. Companies that invest in their local communities, which is a key component of their business model, as well as a positive impact on the environment. This, in turn, has a positive impact on the economy. This, in turn, has a positive impact on the environment. Companies that invest in their local communities, as well as their employees. This, in turn, has a positive impact on the economy. Companies that invest in their local communities, as well as their employees and their communities. This, in turn, has a positive impact on the economy. This, in turn, has a positive impact

on the environment. This, in turn, has a positive impact on the environment. This, in turn, has a positive impact on the economy. Companies that invest in their local

Analysis. The boundary between the model’s genuine learned content and the looping tail is visible in the output. The first four sentences — roughly 80–100 tokens — constitute a coherent structural transformation: the formulaic “In this essay, we will explore...” opening is absent, the “On one hand / on the other hand” scaffolding is broken, and transition phrase density is reduced. After that, “This, in turn, has a positive impact on...” repeats approximately twelve times without adding propositional content. The `min_length` constraint is being satisfied entirely by looping.

Comparing Example 1 (vaccination) and Example 2 (CSR) directly: the same model, trained on the same 40 pairs, produces qualitatively different behavior depending solely on whether it reaches its natural stopping point before the token budget is exhausted. When it does (vaccination), the output is useful. When it does not (CSR with `min_length`), the output degrades. This is not an argument against T5-base for all rewriting tasks—it is a precise statement about the mismatch between T5’s natural operating range and the length-preservation requirement. A decoder-only backbone with an explicit length reward would change the location of that boundary.